

From the Calculus of Constructions to CIC

Lecture 17

Section 1

Part I: Review — Why Sorts Exist

Last time we introduced λC , the Calculus of Constructions:

$$\mathcal{R}_{\lambda C} = \{(*, *), (\Box, *), (\Box, \Box), (*, \Box)\}$$

The four rules allow Π -types that mix quantification over terms, types, predicates, and type operators — a higher-order predicate logic inside λ -calculus.

We saw that λC can *state* the induction principle for Church numerals but cannot *inhabit* it. Today we review the architecture of λC — focusing on the role of sorts — and then extend it to the **Calculus of Inductive Constructions** (CIC).

The question we never asked

Throughout the course, we used two sorts: $*$ (the sort of types) and $\square$ (the sort of kinds). But we never paused to ask:

Why do we need sorts at all?

The simply typed λ -calculus had no sorts. Types were just syntactic expressions written after a colon, and they classified terms. Why did anything change?

STLC: types are external decoration

In STLC, types and terms are **separate syntactic categories**. You write types like $A \rightarrow B$ and terms like $\lambda x:A. t$, but types never appear where terms go, and terms never appear where types go.

Syntactic layer	Expressions
Terms	$x, \lambda x:A. t, t \ s$
Types	$\iota, A \rightarrow B$

These are two disjoint grammars. The typing judgment $\Gamma \vdash t : A$ relates them, but nothing asks “what is the type of $A \rightarrow B$?” The question does not arise because types are not terms — they cannot appear in a position where typing is needed.

System F: types enter the term world

In System F, we added $\Lambda\alpha.t$ (type abstraction) and $t \llbracket A \rrbracket$ (type application). Now a term like $\Lambda\alpha.\lambda x:\alpha.x$ **binds a type variable**. The type of this term is $\forall\alpha.\alpha \rightarrow \alpha$.

But notice what happened: we now write $\Pi\alpha:?.\alpha \rightarrow \alpha$. What goes in the “?”? We need to say that α ranges over *types*. But in STLC, types had no classifier.

System F: types enter the term world

In System F, we added $\Lambda\alpha.t$ (type abstraction) and $t \llbracket A \rrbracket$ (type application). Now a term like $\Lambda\alpha.\lambda x:\alpha.x$ **binds a type variable**. The type of this term is $\forall\alpha.\alpha \rightarrow \alpha$.

But notice what happened: we now write $\Pi\alpha:?.\alpha \rightarrow \alpha$. What goes in the “?”? We need to say that α ranges over *types*. But in STLC, types had no classifier.

The moment types become things we quantify over, we need a name for the “collection of all types.” That name is $*$.

Suppose you are designing System F and you try to avoid introducing *. You might write:

$$\Pi\alpha. \alpha \rightarrow \alpha$$

without annotating α . But now the type checker faces a problem: when it sees α in the body $\alpha \rightarrow \alpha$, it needs to know that α is a type (so $\alpha \rightarrow \alpha$ is a well-formed type) rather than a term (which would make $\alpha \rightarrow \alpha$ nonsensical). In STLC this was enforced by separate grammars. In System F, the grammars merged.

You are forced to annotate: $\Pi\alpha:*. \alpha \rightarrow \alpha$. The sort $*$ tells the type checker: “ α is a type-level variable.” You have rediscovered sorts.

From $*$ to $\Box$: the next step

With $*$ in hand, the expression $\Pi\alpha:*. \alpha \rightarrow \alpha$ is well-formed. But now we can ask: what is the type of the *whole expression* $\Pi\alpha:*. \alpha \rightarrow \alpha$?

It classifies terms (like $\Lambda\alpha. \lambda x:\alpha. x$), so it should be a type. That is, $\Pi\alpha:*. \alpha \rightarrow \alpha : *$. In System F, this works: every $\forall$ -type is itself a type.

From $*$ to $\square$: the next step

With $*$ in hand, the expression $\Pi\alpha:*. \alpha \rightarrow \alpha$ is well-formed. But now we can ask: what is the type of the *whole expression* $\Pi\alpha:*. \alpha \rightarrow \alpha$?

It classifies terms (like $\Lambda\alpha. \lambda x:\alpha. x$), so it should be a type. That is, $\Pi\alpha:*. \alpha \rightarrow \alpha : *$. In System F, this works: every $\forall$ -type is itself a type.

But what about the expression $*$ itself? In the unified syntax of the λ -cube, $*$ is an expression that can appear in Π -binders: $\Pi\alpha:*. \dots$. What is the type of $*$?

If $* : *$, we get Girard's paradox (the type-theoretic analogue of Russell's paradox). The system becomes inconsistent.

The solution: $\square$

To avoid $* : *$, we introduce a second sort: $\square$. The axiom is $* : \square$.

Sort	Classifies	Examples
$*$	types (which classify terms)	$\text{Nat}, \text{Nat} \rightarrow \text{Nat}, \forall \alpha. \alpha \rightarrow \alpha$
$\square$	kinds (which classify types)	$*, * \rightarrow *$

The hierarchy is: terms are classified by types, types are classified by kinds, and $\square$ classifies kinds (including $*$ itself and type-level functions like $* \rightarrow *$).

Could you have avoided $\square$?

In principle, if you are willing to give up System F_ω (no type-level functions), you might try to stop at $*$. System F itself can be presented with just $*$ as a sort and the rule $\forall \alpha : *. A : *$ (impredicativity: polymorphic types are themselves types).

But the moment you want type operators — $\text{List} : * \rightarrow *$, or $\text{Pair} : * \rightarrow * \rightarrow *$ — you need to *classify* them. What is the type of $\lambda \alpha : *. \alpha \rightarrow \alpha$? It takes a type and returns a type, so its “type” should be $* \rightarrow *$. But $* \rightarrow *$ is not a type (it doesn’t classify terms) — it is a **kind**. Kinds live in $\square$.

So sorts arise from a single pressure: when expressions from one level enter the syntax of another, you need a classifier for the upper level.

The PTS perspective

In a **pure type system** (PTS), the architecture is uniform:

- A set $\mathcal{S}$ of **sorts**: $\{*, \Box\}$ for the λ -cube
- A set $\mathcal{A}$ of **axioms**: $\{* : \Box\}$
- A set $\mathcal{R}$ of **rules**: subsets of $\mathcal{S} \times \mathcal{S}$

A rule $(s_1, s_2) \in \mathcal{R}$ allows forming $\Pi x:A. B : s_2$ when $A : s_1$ and $B : s_2$.

The PTS perspective (continued)

The entire λ -cube is parameterized by $\mathcal{R}$:

Rule	Meaning	Example
$(*, *)$	terms depend on terms	$\text{Nat} \rightarrow \text{Nat}$
$(\square, *)$	terms depend on types	$\forall \alpha : * . \alpha \rightarrow \alpha$
$(\square, \square)$	types depend on types	$\lambda \alpha : * . \alpha \rightarrow \alpha : * \rightarrow *$
$(*, \square)$	types depend on terms	$\text{Vec } \alpha \ n \text{ with } n : \text{Nat}$

λC selects all four. The PTS framework shows that the entire cube is just different choices of $\mathcal{R}$ within the same two-sort, one-axiom setup.

What $\Box$ cannot see

There is an important limitation. With $\mathcal{S} = \{*, \Box\}$ and $\mathcal{A} = \{* : \Box\}$, we have:

- $\text{Nat} : *$ and $* : \Box$ — so Nat has a type, and $*$ has a type
- But $\Box$ has **no type**. There is no sort above $\Box$.

This means we cannot quantify over *sorts*. We cannot write $\Pi s:?. \dots$ where s ranges over $*$ and $\Box$. The hierarchy is **capped at two levels**.

Section 2

Part II: The Universe Hierarchy

Problem 1: $\Box$ is a dead end

In λC , type operators like $\text{List} : * \rightarrow *$ live in $\Box$. The rule $(\Box, \Box)$ lets us form kind expressions like $* \rightarrow *$, and everything works.

But suppose we want to **quantify over kinds**. For instance, we might want to write a definition that is polymorphic in a kind κ :

$$\Pi_{\kappa: ?}. \Pi F: \kappa. \dots$$

What goes in the “?”? We need a classifier for $\Box$. But $\Box$ has no type — there is no sort above it.

Problem 1: $\Box$ is a dead end (example)

We can define the polymorphic identity for types:

$$\text{id} : \prod \alpha : * . \alpha \rightarrow \alpha$$

This quantifies over $*$, which is fine because $* : \Box$ and the rule $(\Box, *)$ allows it. Now suppose we want the analogous identity **at the kind level** — something that takes a kind like $*$ or $* \rightarrow *$ and returns it:

$$\text{id}_{\Box} : \prod \kappa : \Box . \kappa \rightarrow \kappa$$

To form this Π -type, we need $\Box$ to be classified by some sort s , so that the rule $(s, \Box)$ applies. But $\Box$ has no classifier. We cannot even *write* this type, let alone inhabit it.

Problem 2: Girard's paradox

We said $* : *$ is dangerous — it allows encoding Russell's paradox, making the system inconsistent (Girard, 1972). The two-sort system with $* : \square$ avoids this.

But the two-sort system creates its own tension. In λC as we defined it, a polymorphic type like $\forall \alpha : *. \alpha \rightarrow \alpha$ lives in $*$. This is **impredicative**: the quantification ranges over all of $*$, including the type being defined.

Problem 2: impredicativity at higher levels

Impredicativity is essential for encoding logic (it gives us $\perp = \forall \alpha: * . \alpha$). But in CIC, inductive types bring **pattern matching**, and that makes impredicativity dangerous at higher levels.

Why pattern matching + impredicativity is dangerous

Suppose Type_0 were impredicative, as $*$ is in λC . Define:

$$\begin{array}{ll} \text{Wrap} & : \text{Type}_0 \quad (\text{impredicativity keeps it in } \text{Type}_0) \\ \text{mk} & : \text{Type}_0 \rightarrow \text{Wrap} \end{array}$$

The constructor `mk` lets us **store** any type inside a `Wrap` value. The recursor lets us **pattern-match** to extract it back out:

$$\text{unwrap} : \text{Wrap} \rightarrow \text{Type}_0 \quad \text{unwrap} (\text{mk } A) \equiv A$$

The paradox

Now $\text{Wrap} : \text{Type}_0$, so Wrap is itself a member of Type_0 . We can store it inside itself: $\text{mk Wrap} : \text{Wrap}$. And we can extract it back: $\text{unwrap} (\text{mk Wrap}) \equiv \text{Wrap}$.

The universe Type_0 embeds into one of its own members and back. From this one can reconstruct Girard's paradox — essentially Russell's “set of all sets that don't contain themselves,” internalized at the type level.

How predicativity solves this

With predicativity, the constructor $\text{mk} : \text{Type}_0 \rightarrow \text{Wrap}$ forces $\text{Wrap} : \text{Type}_1$, because $\text{Type}_0 : \text{Type}_1$ appears in the constructor's type. Pattern matching still works — $\text{unwrap} (\text{mk } A) \equiv A$ — but now Wrap lives *above* Type_0 , not inside it. The self-reference is broken.

Why λC and Prop are safe

λC : there are no inductive types, so there is no pattern matching. You can quantify over $*$ inside Π -types, but you cannot *store* a type in a data structure and *extract* it back. Without pattern matching, impredicativity is harmless.

Prop in **CIC**: you *can* define inductive types in Prop, but the **large elimination restriction** blocks pattern matching from Prop into Type. You cannot write $\text{unwrap} : \text{Wrap} \rightarrow \text{Type}_0$ if $\text{Wrap} : \text{Prop}$. Without pattern matching into Type, the dangerous extraction step is blocked, and impredicativity is safe.

The universe hierarchy

CIC replaces the two sorts $\{*, \square\}$ with an infinite tower:

$$\text{Prop} : \text{Type}_0 : \text{Type}_1 : \text{Type}_2 : \dots$$

Universe	Contains	Predicativity
Prop	propositions	impredicative
Type ₀	“small” types (Nat, Bool, ...)	predicative
Type ₁	types that mention Type ₀	predicative
Type _{<i>i</i>}	types that mention Type _{<i>i</i>-1}	predicative

The key typing rule for predicative universes: if $A : \text{Type}_i$ and $B : \text{Type}_j$, then $\Pi x:A. B : \text{Type}_{\max(i,j)}$. The universe of the Π -type is the maximum of the component universes.

Examples: what lives where (1/2)

Expression	Universe
True, False, $0 = 0$, $\forall n:\text{Nat}. n = n$	Prop
Nat, Bool, $\text{Nat} \rightarrow \text{Nat}$, List Nat, Prop	Type ₀
Type ₀ , $\text{Type}_0 \rightarrow \text{Type}_0$, $\Pi \alpha:\text{Type}_0. \alpha \rightarrow \alpha$	Type ₁
Type ₁ , $\text{Type}_1 \rightarrow \text{Type}_0$	Type ₂

Notice $\text{Prop} : \text{Type}_0$ (propositions are small types) and $\text{Type}_i : \text{Type}_{i+1}$ (each universe lives in the next one). The type of List viewed as a function $\text{Type}_0 \rightarrow \text{Type}_0$ lives in Type_1 .

Examples: what lives where (2/2)

The predicative rule $\Pi x:A. B : \text{Type}_{\max(i,j)}$ in action:

Type	Why	Lives in
$\text{Nat} \rightarrow \text{Bool}$	$\text{Nat} : \text{Type}_0, \text{Bool} : \text{Type}_0$	Type_0
$\Pi \alpha : \text{Type}_0. \alpha \rightarrow \alpha$	domain is $\text{Type}_0 : \text{Type}_1$	Type_1
$\forall n : \text{Nat}. n = n$	$\text{Nat} : \text{Type}_0$, but $n = n : \text{Prop}$	Prop

Notice the last line: $\forall n : \text{Nat}. n = n$ quantifies over $\text{Nat} : \text{Type}_0$ but stays in Prop because Prop is impredicative. If Prop were predicative, this would be pushed up to Type_0 and would no longer be a proposition.

Impredicativity of Prop

Prop is special. The rule is: if $A : \text{Type}_i$ (for any i) and $B : \text{Prop}$, then $\Pi x:A. B : \text{Prop}$. The quantification over a large domain does **not** push the result up — it stays in Prop.

This is what lets us write $\perp \equiv \Pi \alpha:\text{Prop}. \alpha$ and $\forall x:\text{Nat}. P\ x$ and have these remain propositions regardless of how large the domain is.

Proof irrelevance and large elimination

Why is impredicativity safe for Prop but not for Type? Because Prop blocks the dangerous step: pattern matching into Type.

In Lean 4, if $p : \text{Prop}$ and $h_1\ h_2 : p$, then h_1 and h_2 are **definitionally equal**. You cannot write a function that inspects which proof you have and returns different types — the **large elimination restriction** forbids it. So the unwrap function from the Girard argument cannot be defined when $\text{Wrap} : \text{Prop}$.

(Exception: inductive types in Prop with at most one constructor carrying no data, like And or True, can be eliminated into Type — there is nothing to distinguish anyway.)

Design tradeoff: Lean vs. Agda

	Lean 4 (CIC)	Agda (MLTT)
Hierarchy	$\text{Prop}, \text{Type}_0, \text{Type}_1, \dots$	$\text{Set}_0, \text{Set}_1, \text{Set}_2, \dots$
Proof irrelevance	yes (in Prop)	no (by default)
Impredicativity	Prop is impredicative	fully predicative
Extract witnesses?	no (proofs erased)	yes (proofs are data)

Design tradeoff: consequences

Lean: proof irrelevance makes Prop safe to be impredicative. Proofs carry no computational content and can be erased at runtime. The cost: you cannot extract a witness from an existence proof.

Agda: two proofs of the same proposition are different terms; you can pattern-match on them and extract computational content. The cost: the entire hierarchy must be predicative.

Agda does offer an optional Prop universe (`--prop`) with proof irrelevance, but it is opt-in.

Comparison with λC

	λC	CIC
Sorts	$*, \Box$	$\text{Prop}, \text{Type}_0, \text{Type}_1, \dots$
Axioms	$* : \Box$	$\text{Prop} : \text{Type}_0, \text{Type}_i : \text{Type}_{i+1}$
Impredicative?	$*$ is impredicative	only Prop is impredicative
Cap	$\Box$ has no type	every Type_i has Type_{i+1}

The infinite hierarchy resolves Problem 1: every sort has a sort above it. The identity function from the earlier example can now be written once, polymorphic in a universe level u , and applied at every level.

Section 3

Part III: Inductive Types

λC has only one intro/elim pair

Think about λC from the Curry–Howard perspective. It has exactly one way to introduce a type and one way to eliminate it:

Type	Introduction	Elimination
$\Pi x:A. B$	$\lambda x:A. t$	application $f\ a$

That is all. Every type in λC is a Π -type, and every term is built from λ -abstractions and applications. Church encodings simulate data by cleverly using Π -types, but they are still just Π -types — there is no pattern matching.

Inductive types = new intro/elim rules

An inductive definition declares **new introduction rules** (constructors) for a new type. The system then automatically generates the corresponding **elimination rule** (the dependent recursor), whose type is always the induction principle.

Type	Introduction	Elimination
$\Pi x:A. B$	λ -abstraction	application
Nat	zero, succ	Nat.rec (induction)
List α	nil, cons	List.rec (list induction)
Eq α a b	refl	Eq.rec (substitutivity)

Pattern matching is just using the elimination rule.

What an inductive declaration specifies

An inductive type is specified by:

- A **name** and a type (which universe it lives in)
- A finite list of **constructors** (introduction rules), each with a specified type

From this, the system automatically derives:

- The **dependent eliminator** (recursor), whose type is the induction principle
- **Computation rules** (ι -reductions) that fire when the eliminator meets a constructor

Example: natural numbers

Declaration	Type
Nat	: Type_0
zero	: Nat
succ	: $\text{Nat} \rightarrow \text{Nat}$

From this declaration, CIC generates the recursor:

$$\begin{aligned} \text{Nat.rec} : & \prod P : (\text{Nat} \rightarrow \text{Type}_u). \\ & P \text{ zero} \rightarrow (\prod m : \text{Nat}. P \ m \rightarrow P \ (\text{succ } m)) \\ & \rightarrow \prod n : \text{Nat}. P \ n \end{aligned}$$

The recursor vs. System T

Compare with System T's recursor:

$$R : C \rightarrow (\text{Nat} \rightarrow C \rightarrow C) \rightarrow \text{Nat} \rightarrow C$$

There, C was a fixed type. In Nat.rec , the return type $P\ n$ **depends on** n : it can be a different type for each n . This is exactly the induction principle.

The computation rules (ι -reduction)

Nat.rec reduces by matching on the last argument:

$$\text{Nat.rec } P \ p_0 \ s \ \text{zero} \equiv p_0$$

$$\text{Nat.rec } P \ p_0 \ s \ (\text{succ } n) \equiv s \ n \ (\text{Nat.rec } P \ p_0 \ s \ n)$$

These are called ι -reductions (ι for “inductive”). They sit alongside β -reduction and δ -reduction in CIC’s reduction machinery.

Why ι -reduction matters

ι -reduction fires only when the recursor meets a **constructor**. If the argument is a variable, the term is stuck (a **neutral term**).

This is what Church encodings cannot do. A Church numeral is an opaque λ -term; a constructor like `succ  $n$` has *visible structure* that the recursor can inspect.

Example: lists

Declaration	Type
List α	$: \text{Type}_u$ (for $\alpha : \text{Type}_u$)
nil	$: \text{List } \alpha$
cons	$: \alpha \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$

The generated recursor:

$$\begin{aligned} \text{List.rec} : & \prod P : (\text{List } \alpha \rightarrow \text{Type}_v). \\ & P \text{ nil} \\ & \rightarrow (\prod a : \alpha. \prod l : \text{List } \alpha. P l \rightarrow P (\text{cons } a l)) \\ & \rightarrow \prod l : \text{List } \alpha. P l \end{aligned}$$

To prove P for all lists, provide a proof for the empty list and a proof that P is preserved by consing.

Example: propositional equality

Declaration	Type
$\text{Eq } \alpha \ a$	$: \alpha \rightarrow \text{Prop} \quad (\text{for } \alpha : \text{Type}_u, a : \alpha)$
refl	$: \text{Eq } \alpha \ a \ a$

There is only one constructor, refl , and it only produces $\text{Eq } \alpha \ a \ a$ (not $\text{Eq } \alpha \ a \ b$ for arbitrary b).

The recursor for equality

$$\text{Eq.rec} : \prod P : (\alpha \rightarrow \text{Prop}).$$
$$P\ a \rightarrow \prod b : \alpha. \text{Eq}\ \alpha\ a\ b \rightarrow P\ b$$

Read this as: if P holds for a , and $a = b$, then P holds for b . This is **substitutivity** — the elimination form of equality.

Compare with the Leibniz encoding from last lecture: the idea is the same, but now equality is a primitive inductive type, not an encoding.

Inductive equality vs. Leibniz equality

Last time we *defined* $\text{Eq } \alpha \ x \ y \equiv \prod P:(\alpha \rightarrow *). P \ x \rightarrow P \ y$ (Leibniz).
Now Eq is a primitive inductive type with a single constructor refl .

What changes?

- **Leibniz:** equality is a Π -type. Its only elimination rule is function application. There is no way to prove that refl is the *only* proof of $\text{Eq } \alpha \ x \ x$ — you cannot case-split on a Π -type.

Inductive equality vs. Leibniz equality (continued)

- **Inductive:** equality has its own introduction rule (refl) and its own elimination rule (Eq.rec). The elimination rule knows that refl is the *only* constructor, so pattern matching on a proof of $\text{Eq } \alpha \ a \ b$ forces $b \equiv a$. This is strictly more powerful.

In practice, inductive equality gives us the **J-rule** (the recursor above), from which symmetry, transitivity, congruence, and transport all follow.

The general pattern

Every inductive type declaration follows the same recipe:

- 1 **Declare** the introduction rules (constructors) for the new type.
- 2 **Generate** the elimination rule (dependent recursor). Its type is always the type's *induction principle*.
- 3 **Add computation rules**: the elimination rule applied to an introduction rule reduces to the corresponding branch.

Compare with λC : there, the only elimination rule is function application, which cannot inspect the structure of its argument.

What inductive types add to the reduction system

λC has β -reduction (and δ if we allow definitions). CIC adds:

Reduction	When it fires	Example
β	$(\lambda x. t) s$	$(\lambda x. x) 3 \rightarrow 3$
δ	unfold definition	$\text{double } n \rightarrow n + n$
ι	recursor meets constructor	$\text{rec } \dots (\text{succ } n) \rightarrow s \ n \ (\dots)$

Strong normalization of CIC means all three forms of reduction terminate on well-typed terms. This is harder to prove than for λC alone, because ι -reduction introduces recursive computation.

Section 4

Part IV: The Positivity Condition

Not every inductive definition is safe

Consider this attempted inductive type:

$$\begin{array}{ll} \text{Bad} & : \text{Type}_0 \\ \text{intro} & : (\text{Bad} \rightarrow \text{Bad}) \rightarrow \text{Bad} \end{array}$$

The constructor takes a function $\text{Bad} \rightarrow \text{Bad}$. This is dangerous: we can extract this function and apply it to itself, encoding a fixed-point combinator. The system becomes inconsistent (non-termination $\rightarrow$ every type is inhabited).

CIC imposes a **strict positivity condition**: in every constructor, the type being defined may only appear in **strictly positive** positions. Informally:

- **Positive**: $\dots \rightarrow T$ (the type T appears as the final target, or nested under positive type formers)
- **Negative**: $T \rightarrow \dots$ (the type T appears to the left of an arrow)
- **Strictly positive**: T does not appear to the left of any arrow in the constructor's type, even nested

Strict positivity: examples

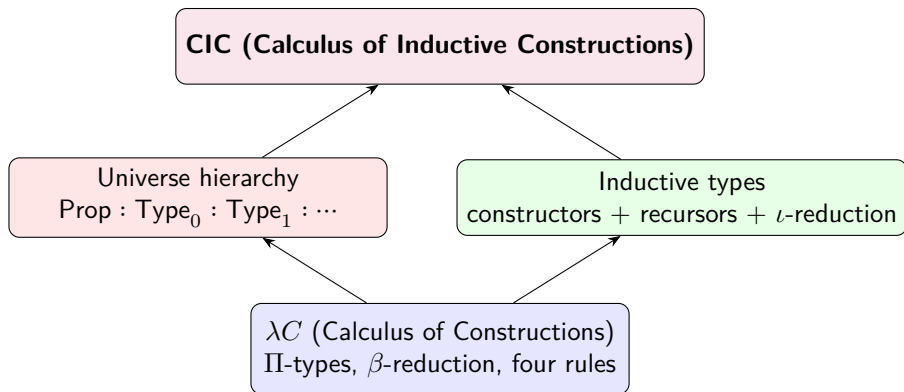
$\text{succ} : \text{Nat} \rightarrow \text{Nat}$	✓	Nat appears positively
$\text{cons} : \alpha \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$	✓	List α appears positively
$\text{intro} : (\text{Bad} \rightarrow \text{Bad}) \rightarrow \text{Bad}$	✗	Bad appears negatively

The strict positivity condition ensures that every inductive type has a well-founded structure: you cannot build circular data, and the recursor always terminates.

Section 5

Part V: Putting It All Together

$\text{CIC} = \lambda C + \text{universes} + \text{inductive types}$



What each piece contributes

Component	Provides	Without it
λC	dependent types, polymorphism, higher-order quantification	cannot state induction, cannot mix terms and t
Universes	infinite hierarchy, Prop vs. Type distinction	$\Box$ has no type; cannot quantify over so
Inductive types	constructors, pattern matching, ι -reduction, induction	only Church encodings; no induction principle

Without λC , we cannot even state dependent types. Without the hierarchy, the system is either inconsistent or capped. Without inductive types, we have no induction.

CIC as the foundation of proof assistants

Proof assistant	Foundation
Coq (1989–)	CIC (original target of CIC)
Lean 4 (2021–)	CIC with modifications
Agda	related (intensional Martin-Löf type theory)

When you write `inductive Nat where | zero | succ (n : Nat)` in Lean, you are using CIC's inductive type mechanism. When Lean generates `Nat.rec`, it is generating the dependent eliminator. When you write `match n with | zero => ... | succ m => ...`, it compiles to applications of the recursor with ι -reduction.

The road so far

System	Key addition	Logic	
STLC	$\lambda x:A. t$	propositional	9
System T	Nat, R	(Dialectica)	1
System F	$\Lambda \alpha. t$	2nd-order prop.	1
F_ω	$\lambda \alpha:\kappa. T$	higher-order prop.	1
λP	types on terms	1st-order pred.	1
λC	all four rules	higher-order pred.	1
CIC	universes + inductive types	higher-order pred. + induction	1

Section 6

Summary

What we covered (1/2)

- ① **Why sorts exist:** types entering term syntax forces classifiers; $*$ classifies types, $\square$ classifies kinds, and the pressure does not stop.
- ② **Rediscovering the hierarchy:** each time you want to quantify over a level, you need a level above it. Two levels $(*, \square)$ cap the hierarchy; CIC uncaps it.
- ③ **The universe hierarchy:** $\text{Prop} : \text{Type}_0 : \text{Type}_1 : \dots$ with Prop impredicative and the Type_i predicative.

What we covered (2/2)

- ④ **Inductive types**: declared by constructors; the system generates a dependent eliminator (induction principle) and ι -reduction rules.
- ⑤ **Strict positivity**: not every inductive definition is safe; CIC restricts constructors to avoid non-termination.
- ⑥ **CIC = λC + universes + inductive types**: the foundation of Coq and Lean.

Next lecture: CIC in depth.

We study the general scheme for inductive types (indexed families, mutual and nested inductives), the distinction between Prop and Type (proof irrelevance, large elimination restrictions), and we work through extended examples in Lean 4.